

Project Report On

**Multi-Agent AI Data Lakehouse Platform for
Betting Site Data Intelligence**

Submitted in partial fulfilment for the award of
Post Graduate Certificate Programme in Big Data Analytics

Guided by:
Ms. Krishnaveni

Presented by:

Ms. Riya Modi	PRN No. 260250325032
Mr. Ilapavuluri Sesha Satya Sri Charan	PRN No. 260250325017
Mr. Prasad Pandurang Gautre	PRN No. 260250325029
Mr. Niraj Sunil Kadam	PRN No. 260250325025

Centre for Development of Advanced Computing (C-DAC), Hyderabad

ACKNOWLEDGEMENT

This project "Multi-Agent AI Data Lakehouse Platform for Betting Site Data Intelligence" was a valuable learning experience, combining data engineering, machine learning and generative-AI techniques into a single working pipeline. We are pleased to submit this work as part of our academic requirements.

We are grateful to Ms. Krishnaveni for the guidance and support provided throughout the course of this project. Her feedback helped us navigate the technical challenges involved in building a multi-stage, multi-agent data platform.

We also thank the Centre for Development of Advanced Computing (C-DAC), Hyderabad and the faculty/coordinators of the Post Graduate Certificate Programme in Big Data Analytics for providing the resources, infrastructure and encouragement needed to complete this project.

From:

Ms. Riya Modi

Mr. Sri Charan

Mr. Prasad Gautre

Mr. Niraj Kadam

TABLE OF CONTENTS

1. Introduction	4
2. Literature Review	5
2.1 Web Scraping and Browser Automation	5
2.2 Streaming Ingestion and Object Storage	5
2.3 Distributed Processing and Feature Engineering	5
2.4 Machine Learning for Classification & Anomalies	5
2.5 Semantic Search, Embeddings and RAG	6
2.6 Positioning of This Project	6
3. System Requirements	7
3.1 Functional Requirements	7
3.2 Non-Functional Requirements	7
3.3 Hardware Requirements	7
3.4 Software Requirements	8
4. System Design & Architecture	9
4.1 Architectural Style	9
4.2 Technology Stack Summary	9
5. Detailed Workflow	11
5.1 Agent 1 - Data Collection Agent	11
5.2 Agent 2 - Streaming & Storage Agent	11
5.3 Agent 3 - Processing & Analytics Agent	12
5.4 Agent 4 - Machine Learning Agent	12
5.5 Agent 5 - Semantic Retrieval Agent	12
5.6 Agent 6 - RAG & Generation Agent	13
5.7 Agent 7 - Verification Agent	13
5.8 Agent 8 - Presentation Agent	13
6. Module Descriptions & Pseudo-code	15
7. Data Flow & Storage Artifacts	17
8. Implementation Details	18
9. Results & Observations	20
10. Advantages, Limitations, and Risks	26
11. Future Scope	28
12. Conclusion	29
13. References	30
Appendix A: Code Listings (Selected Excerpts)	31

1. INTRODUCTION OF PROJECT

Online betting and payment platforms generate large volumes of semi-structured transaction data spread across many payment methods, currencies and page templates. Understanding this data - which categories of payment dominate, which records look unusual, and how a user can ask plain-language questions about it - requires more than a single script. It requires a pipeline of cooperating, specialised components, each responsible for one stage of the journey from raw web page to grounded natural-language answer.

This project implements a Multi-Agent AI Data Lakehouse Platform for Betting Site Data Intelligence. The word "agent" is used in the software sense: each stage of the system owns a well-defined responsibility (scraping, streaming, storage, cleaning, modelling, retrieval, generation, verification and presentation) and communicates with the next stage through lightweight, well-defined artifacts - JSON records, Kafka topics, object-storage buckets, feature tables, vector indexes and evidence traces. Together these agents form a lakehouse-style platform: a central, cleaned data layer (the "lake") that is analysed with both classical machine learning and a retrieval-augmented large language model.

Payment information is collected from four betting websites (Melbet, 22play, 1xBet and 10Cric) using browser automation, because payment pages are dynamic, iframe-heavy and require real interaction to reveal UPI, bank and crypto details. Once collected, the raw JSON is streamed through Apache Kafka, landed in MinIO object storage, and processed with Apache Spark to produce a clean, unified dataset of 161 payment records. Three machine-learning models - Random Forest, Isolation Forest and K-Means - are then applied for classification, anomaly detection and behavioural clustering respectively.

On top of the cleaned data, the platform adds a semantic-search and Retrieval-Augmented Generation (RAG) layer: payment records are converted into short natural-language descriptions, embedded with a Sentence Transformer, indexed with FAISS, and retrieved to ground a locally-run Llama 3 model (via Ollama and LangChain) so that answers to user questions are restricted to actual evidence in the dataset rather than free invention. A Python and LLM-based verification step checks the generated answer against the retrieved evidence before it is shown to the user. Finally, a Streamlit interface exposes the entire pipeline - dashboards, ML results and a question-answering assistant - to an end user.

2. LITERATURE REVIEW

2.1 Web Scraping and Browser Automation

Traditional scraping libraries such as BeautifulSoup and requests are adequate for static HTML but fail on modern payment pages that render content dynamically through JavaScript, load values inside iframes, or require a button click to reveal a QR code or wallet address. Browser-automation tools such as Selenium (used in this project) drive a real Chrome instance, allowing the scraper to click, wait for dynamic content, switch into iframes and capture the fully rendered page before parsing it with BeautifulSoup. Lighter-weight alternatives such as nodriver and Playwright were evaluated during development; nodriver was experimented with but ultimately not used in the four final scraper implementations, and Playwright was not used at all in this project.

2.2 Streaming Ingestion and Object Storage

Apache Kafka is a widely used distributed streaming platform that decouples data producers from consumers, allowing scraped records to be ingested reliably even if downstream consumers are temporarily slow or unavailable. MinIO is an S3-compatible object store frequently used as the storage layer of open-source data-lake and lakehouse architectures because it allows engines such as Apache Spark to read data directly through the S3A connector, without depending on a proprietary cloud provider.

2.3 Distributed Processing and Feature Engineering

Apache Spark (via PySpark) is a distributed processing engine well suited to cleaning, transforming and aggregating semi-structured JSON at scale. In this project, Spark is used to normalise inconsistent fields, derive a reliable payment_category label from free-text payment_method values using keyword rules, and produce the cleaned dataset used for downstream analytics and modelling.

2.4 Machine Learning for Classification, Anomaly Detection and Clustering

Random Forest is an ensemble of decision trees that is robust to noisy, nonlinear feature interactions and provides interpretable feature-importance scores, making it a natural choice for payment-category classification. Isolation Forest is an unsupervised technique that isolates anomalous points by recursively partitioning the feature space, requiring fewer partitions to isolate points that are structurally different from the majority - making it well suited to flagging unusual payment records without labelled anomaly examples. K-Means is a classical unsupervised clustering algorithm that groups records by similarity in feature space and is commonly evaluated using the Silhouette Score, which measures how well-separated the resulting clusters are.

2.5 Semantic Search, Embeddings and RAG

Dense vector embeddings produced by Sentence Transformer models (such as all-MiniLM-L6-v2) allow semantic - meaning-based - search, in contrast to exact keyword matching. FAISS (Facebook AI Similarity Search) provides efficient nearest-neighbour search over such embeddings and is widely used as the retrieval backbone of Retrieval-Augmented Generation (RAG) systems. RAG combines a retriever with a generative language model so that the model's output is grounded in retrieved evidence rather than relying purely on parametric memory, which reduces hallucination. LangChain is a common orchestration framework for building such retrieval-then-generation pipelines, and Ollama provides a way to run open-weight LLMs such as Llama 3 locally rather than through a cloud API.

2.6 Positioning of This Project

Much existing work treats data engineering, machine learning and LLM-based question answering as separate projects. This platform's contribution is to combine all three into a single, cooperating multi-agent pipeline over a domain (betting-site payment data) that is not well served by off-the-shelf tools, and to add an explicit, two-stage evidence-verification step (Python plus an LLM check) that most simple RAG demonstrations omit.

3. SYSTEM REQUIREMENTS

3.1 Functional Requirements

- **Data Collection:** Collect payment-page data (payment method, UPI/bank/crypto details, amounts where present) from four betting websites.
- **Streaming & Ingestion:** Validate and stream scraped JSON records reliably to a storage layer without data loss.
- **Data Cleaning:** Clean and transform raw records into a consistent, analysable dataset with a derived payment_category field.
- **Machine Learning Analytics:** Classify payment records by category, detect anomalous records, and discover behavioural clusters.
- **Natural Language RAG QA:** Allow a user to ask natural-language questions about the payment dataset and receive an evidence-grounded answer.
- **Two-Stage Verification:** Verify that generated answers are supported by retrieved evidence before presenting them.
- **Unified Web UI:** Present dashboards, ML results and the question-answering assistant through a single user interface.

3.2 Non-Functional Requirements

- **Reliability:** Zero data loss between Kafka ingestion, MinIO storage and Spark processing (verified by matching row counts at every stage).
- **Local-First Operation:** The LLM (Llama 3 via Ollama) and embedding model run locally, avoiding dependency on a cloud LLM API.
- **Extensibility:** Each stage (scraper, streaming layer, storage, processing, ML, retrieval, generation, UI) is decoupled so components can be replaced or scaled independently.
- **Explainability:** Feature-importance scores, silhouette scores and evidence traces are exposed rather than hidden inside a black box.

3.3 Hardware Requirements

A workstation capable of running Chrome/Selenium, a local Kafka broker, MinIO, Spark (local or cluster mode) and a local LLM runtime (Ollama) - recommended minimum 16 GB RAM, multi-core CPU, and GPU optional for faster LLM inference.

3.4 Software Requirements

Category	Technology
Language	Python 3
Scraping	Selenium, Chrome WebDriver, BeautifulSoup, Pillow, pyzbar
Streaming	Apache Kafka
Object Storage	MinIO (S3-compatible)
Processing	Apache Spark / PySpark
Machine Learning	scikit-learn (Random Forest, Isolation Forest, K-Means)
Embeddings	Sentence-Transformers (all-MiniLM-L6-v2)
Vector Search	FAISS (IndexFlatL2)
RAG Orchestration	LangChain
Local LLM Runtime	Ollama running Llama 3
User Interface	Streamlit

4. SYSTEM DESIGN & ARCHITECTURE

The platform is organised as a linear pipeline of cooperating agents, each consuming the output of the previous stage and producing a well-defined artifact for the next. The end-to-end flow is summarised below:

```
WEBSITES (Melbet, 22play, 1xBet, 10Cric)
|
Agent 1: Selenium + Chrome WebDriver + BeautifulSoup -> JSON + validation
|
Agent 2: Kafka (topic: payment_raw) -> MinIO (bucket: betting-data)
|
Agent 3: Spark / PySpark -> Cleaning + Feature Engineering + EDA
|
Agent 4: Machine Learning
|-- Random Forest      (classification)
|-- Isolation Forest   (anomaly detection)
`-- K-Means            (clustering)
|
Agent 5: Sentence Transformer (all-MiniLM-L6-v2) -> 384-D Embeddings -> FAISS
|
Agent 6: User Question -> Query Embedding -> FAISS Retrieval -> LangChain RAG -> Ollama / Llama 3
|
Agent 7: Evidence Verification (Python + LLM check)
|
Agent 8: Streamlit Payment Intelligence Interface
```

4.1 Architectural Style

The system follows a multi-agent, pipe-and-filter architecture layered on top of a lakehouse storage pattern. Each agent is a filter with a single responsibility; Kafka and MinIO form the raw/curated storage backbone (the 'lake' layer), while the Spark-cleaned dataset, ML models and FAISS index form an analytical layer on top of it (the 'house' of structured, query-ready data). This separation allows the scraping agents to be re-run independently of the ML agents, and the ML agents to be re-run independently of the RAG layer.

5. DETAILED WORKFLOW

This section describes each of the eight agents in the pipeline, what it does, why that technology was chosen, and the benefit it provides.

5.1 Agent 1 - Data Collection Agent

Responsibility: collect real payment-page information from four betting websites (Melbet, 10Cric, 1xBet, 22play/22Bet).

- Melbet: Selenium + Chrome WebDriver + BeautifulSoup (Dynamic payment flow, iframe interaction, UPI/Bank/Crypto extraction).
- 10Cric: Selenium + Chrome WebDriver + BeautifulSoup + Pillow + pyzbar (Dynamic iframe flow; QR decoding required).
- 1xBet: Selenium + Chrome WebDriver + BeautifulSoup (Manual payment flow, iframe interaction and extraction).
- 22play / 22Bet: Selenium + Chrome WebDriver + BeautifulSoup (Dynamic methods, regex extraction & click fallbacks).

5.2 Agent 2 - Streaming & Storage Agent

JSON semi-structured records preserve scraped payment page information. Apache Kafka moves records from producer to consumer (topic payment_raw) to decouple collection from storage. MinIO S3-compatible object storage lands raw JSON into bucket betting-data. All 161 records passed Kafka delivery verification and MinIO storage with zero data loss.

5.3 Agent 3 - Processing & Analytics Agent

Apache Spark / PySpark reads stored raw JSON, cleans it and performs exploratory data analysis. Raw transaction-detail fields were inconsistent: blank strings were converted to proper nulls, and a reliable payment_category field was derived from free-text payment_method using keyword rules, classifying every one of the 161 records into UPI, Crypto, Wallet or Bank with zero unclassified ('Other') rows.

5.4 Agent 4 - Machine Learning Agent

- Random Forest (Supervised Classification): Predicts payment_category from engineered features.
- Isolation Forest (Unsupervised Anomaly Detection): Scores records that differ from normal patterns.
- K-Means (Unsupervised Clustering): Groups similar records into k clusters.

5.5 Agent 5 - Semantic Retrieval Agent

Each payment row is converted into a short natural-language description. These descriptions are embedded with all-MiniLM-L6-v2 producing 384-dimensional vectors stored in a FAISS IndexFlatL2 index.

5.6 Agent 6 - RAG & Generation Agent

LangChain orchestrates retrieval, context construction and local Llama 3 LLM prompts via Ollama, generating natural language answers restricted to retrieved evidence.

5.7 Agent 7 - Verification Agent

Combines a Python evidence check (verifying concrete items against retrieved records) and an LLM evidence check (evaluating if the draft answer is fully supported).

5.8 Agent 8 - Presentation Agent

Streamlit turns the working pipeline into a simple user interface where a user can view dashboards and ask natural language questions.

6. MODULE DESCRIPTIONS & PSEUDO-CODE

6.1 Scraper Module (per site)

```
function scrape_site(site_config):
    driver = launch_chrome_webdriver()
    driver.get(site_config.payment_page_url)
    switch_to_payment_iframe(driver)
    for method in list_payment_methods(driver):
        click(method)
        html = driver.page_source
        record = parse_with_beautifulsoup(html, site_config.rules)
        if site_config.name == '10Cric' and has_qr_code(html):
            record['qr_payload'] = decode_qr(html) # Pillow + pyzbar
        validate_json(record)
    yield record
```

6.2 Cleaning & Category Derivation Module (Spark)

```
def derive_payment_category(payment_method: str) -> str:
    text = payment_method.lower()
    if any(k in text for k in UPI_KEYWORDS): return 'UPI'
    if any(k in text for k in CRYPTO_KEYWORDS): return 'Crypto'
    if any(k in text for k in WALLET_KEYWORDS): return 'Wallet'
    if any(k in text for k in BANK_KEYWORDS): return 'Bank'
    return 'Other' # 0 records fell into 'Other' across all 161 rows
```

6.3 Machine Learning Module

```
# Classification (leakage-free feature set)
features = ['site_encoded', 'diagnostic_only', 'amount_present', 'ref_url_count', 'html_len', 'plain_text_len']
X_train, X_test, y_train, y_test = train_test_split(df[features], df['payment_category'], test_size=0.2, s
rf = RandomForestClassifier(random_state=42).fit(X_train, y_train)

# Anomaly detection (per-site, corrected version)
for site, group in df.groupby('site_name'):
    iso = IsolationForest(contamination=0.1, random_state=42)
    group['anomaly'] = iso.fit_predict(group[features])

# Clustering
for k in range(2, 8):
    km = KMeans(n_clusters=k, random_state=42).fit(X_scaled)
    silhouette_scores[k] = silhouette_score(X_scaled, km.labels_)
```

6.4 Semantic Description & Embedding Module

```
def describe(record):
    return (f"This is a payment record from {record.site_name}. "
            f"The payment method is {record.payment_method}. "
            f"The payment belongs to the {record.payment_category} category.")

model = SentenceTransformer('all-MiniLM-L6-v2')
embeddings = model.encode([describe(r) for r in records]) # (161, 384)
index = faiss.IndexFlatL2(384)
index.add(embeddings)
```

7. DATA FLOW & STORAGE ARTIFACTS

7.1 Storage Artifacts Summary

Artifact	Location / Form	Produced By	Consumed By
Raw scraped JSON	Kafka topic payment_raw	Agent 1 (Scraper)	Agent 2 (Kafka)
Raw JSON objects	MinIO bucket betting-data	Agent 2 (Kafka)	Agent 3 (Spark)
Cleaned dataset (161 records)	Spark output (CSV/Parquet)	Agent 3 (Spark)	Agent 4 (ML), Agent 5
Trained ML models	RandomForest/IsolationForest/KMeans	Agent 4 (ML)	Streamlit UI
Payment descriptions	In-memory / cached text	Agent 5	Agent 5 (Embeddings)
FAISS index	IndexFlatL2 (384-D, 161 vectors)	Agent 5	Agent 6 (RAG)
Evidence trace	Retrieved records + verification	Agent 6, Agent 7	Agent 8 (Streamlit UI)

8. IMPLEMENTATION DETAILS

8.1 Data Collection Summary

Melbet: 87 records | 22play: 54 records | 1xBet: 12 records | 10Cric: 8 records. Total: 161 records.

8.2 Data Cleaning

Blank strings were converted to proper nulls, and payment_category was derived directly from payment_method text using keyword rules with zero unclassified rows.

9. RESULTS & OBSERVATIONS

9.1 Overall Payment Category Distribution (161 records)

- **Crypto:** 78 records (48.4% share)
- **Wallet:** 47 records (29.2% share)
- **UPI:** 21 records (13.0% share)
- **Bank:** 15 records (9.3% share)

9.2 Random Forest Classification Results

The Random Forest classifier achieved an overall accuracy of 84.8%.

Top predictive features: html_len (57.0%) and plain_text_len (31.8%) together account for ~89% of predictive power.

9.3 Isolation Forest Anomaly Detection Results

Per-site model results: Melbet (9 flagged / 10.3%), 22play (6 flagged / 11.1%), 1xBet (1 flagged / 8.3%), 10Cric (1 flagged / 12.5%). Total: 17 anomalies.

9.4 K-Means Behavioural Clustering Results

k=4 chosen (silhouette 0.846), matching 4 payment categories. Global model silhouette reached 0.932 at k=7 due to 34 duplicate feature records.

10. ADVANTAGES, LIMITATIONS, AND RISKS

10.1 Advantages

- End-to-end coverage spanning data engineering, analytics, ML and generative AI.
- Reliable ingestion with zero data loss between Kafka, MinIO and Spark.
- Grounded, verifiable answers with two-stage Python + LLM verification.

10.2 Limitations

- Small dataset size (161 records).
- Duplicate records (~21%) inflating silhouette scores at higher k.

11. FUTURE SCOPE

- Expand scraper set to additional betting sites beyond 161 records.
- Add automated selector-health checks for each site scraper.

12. CONCLUSION

This project successfully implemented a Multi-Agent AI Data Lakehouse Platform for Betting Site Data Intelligence with Random Forest accuracy of 84.8%, per-site anomaly detection, and zero-hallucination vector RAG QA.

13. REFERENCES

1. Breiman, L. 'Random Forests.' Machine Learning, 45(1), 2001.
2. Liu, F. T., et al. 'Isolation Forest.' IEEE ICDM, 2008.
3. Reimers, N. 'Sentence-BERT.' EMNLP, 2019.
4. Johnson, J., et al. 'Billion-scale similarity search with GPUs.' IEEE, 2019.

A. APPENDIX A: CODE LISTINGS (SELECTED EXCERPTS)

A.1 Payment Category Derivation (Spark)

```
def derive_payment_category(payment_method):
    text = (payment_method or '').lower().strip()
    for keyword_set, label in [(UPI_KEYWORDS, 'UPI'), (CRYPTO_KEYWORDS, 'Crypto'),
                              (WALLET_KEYWORDS, 'Wallet'), (BANK_KEYWORDS, 'Bank')]:
        if any(k in text for k in keyword_set): return label
    return 'Other'
```

A.2 Leakage-Free Random Forest Training

```
FEATURES = ['site_encoded', 'diagnostic_only', 'amount_present', 'ref_url_count', 'html_len', 'plain_text']
X = df[FEATURES]
y = df['payment_category']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, stratify=y, random_state=42)
rf = RandomForestClassifier(random_state=42).fit(X_train, y_train)
print('Accuracy:', accuracy_score(y_test, rf.predict(X_test))) # 0.848
```

A.3 Per-Site Isolation Forest

```
results = []
for site, group in df.groupby('site_name'):
    iso = IsolationForest(contamination=0.1, random_state=42)
    group['anomaly'] = iso.fit_predict(group[FEATURES])
    flagged = (group['anomaly'] == -1).sum()
    results.append((site, len(group), flagged, flagged / len(group)))
```